Informe técnico: revisión del flujo de Primas ARL (freno, SCD y extracción)

Para: equipo de desarrollo, BA y líder técnico

Fecha: 13 de noviembre de 2026

Alcance: flujo de Primas ARL de punta a punta, con foco en el job de Gold, el SCD (Slowly Changing Dimension, dimensión de cambio lento), y en la extracción

Resumen

1. El fix del incidente de producción (la extracción conserva los lotes de respaldo) ya está en `develop`. El freno del SCD (dimensión de cambio lento) funciona: en QA protegió Gold dos veces.
2. **Problema raíz:** el SCD (dimensión de cambio lento) apaga todo lo que no llega en la entrada. Con una entrada parcial, ya sea incremental o de un solo periodo, apaga datos válidos.
3. La corrección del problema raíz es comparar solo dentro del periodo que se carga (SCD-02) y unificar la definición de periodo entre ambientes (ARL-01 y ARL-03). Depende de las respuestas de BA.
4. En `scd.py` hay 9 hallazgos más. Los graves: sobrescribe Gold si falla la lectura, deja duplicados con llaves repetidas o nulas, y no cuadra antes de escribir.
5. En la extracción hay 10 hallazgos. Los graves, confirmados en la simulación del flujo: una reanudación puede mezclar periodos, y los cambios en el origen de registros ya extraídos nunca llegan a Gold.
6. La estructura del código no es mantenible: los jobs son scripts de arriba abajo, sin pruebas, con la cobertura excluida de Sonar y funciones copiadas entre 5 jobs (7 hallazgos COD).
7. En los demás componentes (Silver ARL, certificación, Step Function y validación) quedan 4 puntos, cada uno con su responsable.
8. La simulación del flujo completo (extracción, remediación, Silver ARL y SCD reales) reproduce el incidente, confirma que el fix funciona y confirma los hallazgos (sección 2.1).
9. Siguiente paso: los cambios cortos que protegen los datos y no dependen de nadie (sección 5, punto 1).

Índice

- Repo que se toca
- 1. Descripción general
- 2. Cómo se validó
- 2.1 Resultados de la simulación del flujo
- 3. Problema raíz
- 3.1 Antes del cambio: el incidente
- 3.2 Con el cambio: fix de extracción y freno
- 3.3 Lo que sigue abierto
- 3.4 Corrección del problema raíz
- 4. Otros problemas encontrados
- 4.1 En el SCD (`scd.py`)
- 4.2 En la extracción (`extraccion_estandar_optimized.py`)
- 4.3 Estructura del código y mantenibilidad
- 4.4 En otros componentes
- 5. Orden de trabajo
- 6. Preguntas para BA

Repo que se toca

Repo	Archivo	Qué se hace
`data-datalake-niif-rmd-ext-estandar-glue-job`	`src/jobs/scd.py`	Corrección (secciones 3.4 y 4.1)
`data-datalake-niif-rmd-ext-estandar-glue-job`	`src/jobs/extraccion_estandar_optimized.py`, `remediacion_estandar.py`, `validacion_tecnica.py` y el módulo común nuevo	Corrección (secciones 4.2 y 4.3)
`data-datalake-niif-primas-arl-glue-job`	`src/jobs/silver_primas_arl.py`	ARL-02 (sección 4.4)
Certificación y notificación	Jobs de certificación y notificación	ARL-04 (sección 4.4)
`cfg-datalake-niif-config-s3`	Config de Primas ARL	ARL-01 y ARL-04 (secciones 3.4 y 4.4)

La rama `bugfix/scd-freno-periodo`, creada desde `develop` actualizado (commit `56eaf0c`), es la de la corrección completa en `data-datalake-niif-rmd-ext-estandar-glue-job`. Los otros repos llevan su propia rama con el mismo nombre. Los temas que no son código (DevOps, BA y decisiones) están en la sección 5.

1. Descripción general

El flujo lo ejecuta la Step Function `niif-<amb>-primas-arl`. Cada etapa es un job de Glue que deja su estado en `parametros_extraccion.json` para la siguiente.

```
Extracción → Validación técnica → Remediación → Silver ARL → SCD → Certificación → Notificación
(rmd-ext)    (rmd-ext)            (rmd-ext)     (primas-arl)  (rmd-ext) (certification)  (notification)
```

El fix del incidente de producción, que hace que la extracción conserve los lotes de respaldo, ya está en el repo. El freno del SCD (dimensión de cambio lento) funciona: el 12 de noviembre detuvo en QA dos corridas que iban a apagar todo Gold, una de Auxilios Funerarios (90 de 90) y otra de Primas ARL (2.099 de 2.099). En las dos, Gold quedó intacto.

El informe separa el **problema raíz** (sección 3), que explica tanto el incidente como esas detenciones, de **otros problemas encontrados** en la revisión (sección 4).

2. Cómo se validó

- Lectura del código de cada etapa, en las versiones de `quality` y `develop` del 12 y 13 de noviembre, y de la definición de la Step Function.
- Ejecución del `scd.py` real, sin modificarlo, con PySpark local y casos de prueba controlados.
- Para la extracción: lectura completa del job y de cómo lo usan la remediación, el Silver de ARL y la configuración de las mallas (`cfg-datalake-niif-config-s3`).
- Simulación del flujo completo con los jobs reales, sin modificarlos: extracción, remediación, Silver ARL y SCD (dimensión de cambio lento), encadenados como lo hace la Step Function, con Oracle, S3 y Glue simulados. La validación técnica se simula en `SUCCESS`.
- Evidencia de QA del 12 de noviembre: logs del job y parámetros de las corridas detenidas.

Lo que depende de datos de producción o de una respuesta de negocio aparece marcado como "por confirmar".

2.1 Resultados de la simulación del flujo

Datos de prueba: 10 endosos de 202605 y 10 de 202606, con sus afiliaciones, personas y sucursales.

Escenario	Qué se hizo	Resultado	Qué confirma
Control	202606 con `<=`: corrida base con tope y luego incremental	La incremental trae 2 nuevos, excluye 18 y usa 2 lotes. El SCD apaga 0 y Gold queda con los 20.	El fix de extracción funciona en la cadena completa
Antes del fix	El mismo escenario, con la extracción sin combinar lotes	La incremental usa 1 lote y el SCD iba a apagar 18 de 18. El freno lo detuvo.	Reproduce el incidente de producción. El freno lo evita

Escenario	Qué se hizo	Resultado	Qué confirma
Filtro `=`	202605 y luego 202606 con `PERIODO = 'P'`, como en QA	El SCD iba a apagar 10 de 10. El freno lo detuvo.	Problema raíz (SCD-02 y ARL-01)
Periodo anterior otra vez	202605, 202606 y de nuevo 202605, con `<=`	El SCD iba a apagar 10 de 20 (los de 202606). El freno lo detuvo.	El problema raíz también aparece con `<=`
Reanudación	202605 falla en Oracle a mitad; la siguiente corrida es de 202606	202606 reanudó con el lote de 202605, saltó las tablas y terminó bien, con solo los 10 endosos de 202605	EXT-01
Cambio en el origen	El endoso 12 pasa de 1.200 a 99.999 en Oracle entre dos corridas del mismo periodo	Gold sigue con 1.200	EXT-02
Bronce y Silver base	202605 y luego 202606	Bronce queda solo con el último lote. Silver base guarda los dos y repite los endosos de 202605	EXT-03
Llave en dos lotes	Dos corridas sin exclusión y una tercera con exclusión	La tercera usa 2 lotes de respaldo y Silver lee cada llave dos veces. El `dropDuplicates` de ARL lo oculta	EXT-04 y ARL-02

3. Problema raíz

El SCD (dimensión de cambio lento) **asume que la entrada es la foto completa** de lo que debe existir. Todo lo que está activo en Gold y no llega en la entrada lo apaga (`ACT_REG = 0`). Si la entrada llega parcial, ya sea porque la extracción trajo solo lo nuevo o porque trae un solo periodo, el job apaga datos válidos.

Flujo base:

```
Oracle → Extracción → Silver → SCD (dimensión de cambio lento) → Gold → Certificación
```

3.1 Antes del cambio: el incidente de Primas ARL en producción

```
Oracle        Extracción              Silver         SCD (dimensión de       Gold              Certificación
                                                     cambio lento)
1,2 millones  2ª corrida del mismo    Solo 209       Compara 209 contra      1,2 M apagados    Consulta filtra
de endosos →  periodo: excluye las  → filas (lee  → 1,2 M activos:       → (ACT_REG = 0).  → activos del
              1.203.789 ya extraídas  solo el lote   "los otros no vienen,   Quedan 209         periodo: 0 filas
              y trae 209 nuevas       actual)        ya no existen"          activos           → correo vacío
                    ↑
       El script de prod no guardaba los lotes de respaldo
```

Resumen: la extracción dejó Silver con 209 filas. El SCD (dimensión de cambio lento) creyó que era todo lo que existía y apagó el resto.

3.2 Con el cambio: fix de extracción y freno

```
Oracle        Extracción              Silver         SCD (dimensión de       Gold             Certificación
                                                     cambio lento)
1,2 millones  Excluye las 1.203.789 y Lee los lotes  Recibe la foto          Se agregan       Consulta trae
de endosos →  guarda los lotes de   → de respaldo  → completa. Apaga       → solo los 209  →  los datos del
              respaldo + el actual    + el actual    cerca de 0              nuevos           periodo
                    ↑                                    ↑
              Fix de extracción                Freno: si el SCD (dimensión de cambio lento)
              (corrige la causa)               fuera a apagar más del 20 %, PARA antes de
                                               escribir y Gold queda intacto (red de seguridad)
```

Resumen: el fix evita que Silver llegue incompleto, y el freno protege Gold si algún día vuelve a pasar.

3.3 Lo que sigue abierto: un periodo nuevo apaga los anteriores

Ejemplo de Auxilios Funerarios en QA (12 de noviembre). En Primas ARL pasó lo mismo con 2.099 de 2.099.

```
Oracle        Extracción        Silver             SCD (dimensión de          Gold
                                                   cambio lento)
Periodo       Trae solo         Solo solicitudes   Mira TODOS los activos     Tenía 90 activos
202512    →   202512        →   de 202512      →   de Gold, sin mirar    →   del periodo 202511
                                                   el periodo:                (otro periodo)
                                                   "los 90 de 202511
                                                   no vienen" → los apagaría

                                                         ↓
                                              Freno: 90 de 90 = 100 % > 20 %
                                              → la corrida FALLA, Gold no se toca
```

Resumen: no hay error de datos. El SCD (dimensión de cambio lento) no sabe qué periodo se está cargando y trata los periodos anteriores como desaparecidos. El freno lo detuvo, pero la causa sigue ahí.

La simulación del flujo muestra lo mismo con el filtro `=` (10 de 10) y también con `<=` cuando se vuelve a correr un periodo anterior después de uno más nuevo (10 de 20). Ver sección 2.1.

3.4 Corrección del problema raíz

Con el ajuste, el job compara solo dentro del periodo que se carga:

```
Periodo 202512 → Silver de 202512 → SCD (dimensión de cambio lento) mira SOLO los activos de 202512 en Gold
                                  → los de 202511 no se tocan → Gold conserva los dos periodos
```

ID	Dónde	Qué pasa	Corrección	Quién
SCD-02	`scd.py`, líneas 403 a 410	El apagado por ausencia compara contra todos los activos de Gold, sin mirar el periodo. Evidencia: QA 12-nov (90/90 y 2.099/2.099) y prueba local.	Clave opcional `columna_periodo` en la config de cada malla. Solo se apagan los activos de los periodos que trae la entrada, y el freno mide sobre ese mismo grupo. Sin la clave, el comportamiento es el actual.	Desarrollo, cuando BA responda
ARL-01	Config base del configurador `sc-datalake-niif-primas-arl` y repo `cfg-datalake-niif-config-s3`	El filtro de periodo cambia por ambiente: en QA es `PERIODO = 'P'` (un solo periodo), en producción `PERIODO <= 'P'` (foto acumulada). El configurador solo reemplaza el periodo y conserva el operador. Por eso QA no reproduce lo que pasa en producción.	Definir el operador oficial y alinear la config base de cada ambiente.	BA y desarrollo
ARL-03	Extracción y certificación	La extracción filtra por `PERIODO` de PAR_ENDOSOS y la certificación por `per_proceso_point` (mes de FEC_PROCESO_POINT). En producción (202606) entraron endosos de 202610, y el 91 % de Gold tenía `per_proceso_point` vacío, así que esos registros nunca se certifican.	Que BA defina qué columna manda el periodo de un endoso. Esa misma columna es la que usaría SCD-02 en Primas ARL.	BA

4. Otros problemas encontrados

4.1 En el SCD (dimensión de cambio lento): se corrigen en `scd.py`

ID	Severidad	Qué pasa	Evidencia	Corrección
SCD-01	Crítico	Si Gold existe pero la lectura falla (permiso o error transitorio), el job asume que es la primera carga y sobrescribe Gold con la entrada. Se pierde el historial.	Prueba local: un Gold de 3 filas quedó con 1. Líneas 318 a 353.	Comprobar si la ruta existe. Si existe y no se puede leer, detener sin escribir.
SCD-03	Alto	Una llave repetida en la entrada deja dos registros activos para la misma llave.	Prueba local.	Validar que la llave sea única en la entrada y en Gold activo antes de comparar. Si no lo es, detener con el conteo y una muestra.
SCD-04	Alto	Si una columna de la llave viene nula, el registro se cierra y se vuelve a insertar en cada corrida aunque no cambie. Gold crece sin control. Aplica a mallas con columnas de apertura, como Auxilios Funerarios. Primas ARL no las usa.	Prueba local: 3 corridas con el mismo dato dejaron 3 filas.	Comparación que trate dos nulos como iguales y que clasifique con una marca de origen, en lugar de mirar si la llave es nula.

ID	Severidad	Qué pasa	Evidencia	Corrección
SCD-05	Alto	No hay cuadre antes de escribir, así que los errores de SCD-03 y SCD-04 llegan a Gold sin que nada los detecte.	Código.	Verificar `gold_final = gold_anterior + insertados + nuevas_versiones` y el cuadre de activos. Registrar el resultado en las métricas y detener si no cuadra.
SCD-06	Medio	El job lee y sobrescribe la misma ruta. Un fallo a mitad de la escritura puede dejar Gold incompleto, sin forma de volver atrás.	Código, líneas 501 a 506. No reproducido.	Escribir primero a una ruta de staging, verificar el conteo y luego publicar. Pedir a DevOps versionado de S3 en Gold.
SCD-07	Medio	Los logs informativos (conteos y porcentajes) no aparecen en CloudWatch. Solo sale el error.	QA 12-nov.	Fijar el nivel del logger de forma explícita.
SCD-08	Medio	Cuando el freno se activa, no guarda las métricas ni dice cuántos registros entraron ni de qué periodos son los que se apagarían.	QA 12-nov y prueba local.	Guardar las métricas y ese detalle en el mensaje antes de detener.
SCD-09	Bajo	El cruce entre la entrada y Gold se recalcula unas seis veces en cada corrida, lo que sube el costo de Glue.	Código.	Clasificar una sola vez y reutilizar el resultado.
SCD-10	Bajo	Un cambio de vacío a texto vacío no se detecta. Las columnas marcadas como "alerta" no generan ningún aviso. Una columna que no esté en la config queda con el valor de la primera carga.	Prueba local y código.	Ajustar el cálculo de cambios, contar los cambios en columnas de alerta y validar la config al iniciar.

Se conserva: el freno (umbral del 20 % configurable por malla con `umbral_apagado_pct`), la inicialización de Gold vacío, que una entrada vacía no toque Gold, la validación de los estados previos, el historial tipo 2 y las métricas. La corrección no agrega infraestructura.

4.2 En la extracción: `extraccion_estandar_optimized.py`

La extracción es la que decide qué llega a todo el resto del flujo: escribe Bronce, guarda las llaves de cada segmento y deja en `parametros_extraccion.json` los lotes que leen la remediación, el Silver de cada malla y el SCD (dimensión de cambio lento). Los hallazgos se miran por cómo afectan a esos componentes.

ID	Severidad	Qué pasa	A quién afecta	Evidencia	Corrección
EXT-01	Alto	Si el estado anterior quedó en `FAILED` o `IN_PROGRESS`, la corrida nueva **reanuda** con el lote anterior y salta las tablas ya extraídas, sin comprobar que sea el mismo periodo. Si el periodo cambió, usa datos del periodo anterior. Si otra corrida sigue en curso (`IN_PROGRESS`), se suma a ella.	Silver y Gold reciben datos de otro periodo. Dos corridas a la vez se pisan.	Simulación del flujo (2.1). Código: líneas 1049 a 1097, 498 a 510 y 1169.	Reanudar solo si coinciden periodo y caso y el estado es `FAILED`. Si no coinciden, empezar una corrida nueva y dejarlo en el log.
EXT-02	Alto	La exclusión descarta por llave las filas ya extraídas, aunque su contenido haya cambiado en Oracle. En una nueva corrida del mismo periodo, el valor que llega es el del lote anterior.	El SCD (dimensión de cambio lento) nunca ve esos cambios, y una recertificación puede salir con cifras desactualizadas. Por confirmar con BA si el origen modifica registros ya cargados.	Simulación del flujo (2.1): Oracle con 99.999 y Gold con 1.200. Código: líneas 760 a 762.	Según la respuesta de BA: no excluir al reprocesar el mismo periodo, o comparar el contenido además de la llave.

ID	Severidad	Qué pasa	A quién afecta	Evidencia	Corrección
EXT-03	Medio	Bronce se guarda en una ruta por tabla, compartida entre periodos y entre mallas (17 configuraciones usan la misma base, y `acu_personas_naturales` la usan ARL, Reserva Matemática ARP y RVI). Cada carga sin exclusión **sobrescribe Bronce completo**. Silver base, en cambio, acumula todos los lotes, porque la remediación escribe en modo de recuperación.	Silver base queda como la única copia de los lotes anteriores, crece sin límite y repite llaves entre periodos. Bronce no sirve para reprocesar ni auditar lotes anteriores. Si dos mallas corren a la vez, una puede sobrescribir el Bronce que la otra todavía no ha pasado a Silver (esto último sale del código, no se simuló).	Simulación del flujo (2.1). Código: escritura en Bronce, líneas 1502 a 1519; ruta, líneas 1199 a 1213; remediación, `resolver_modo_escritura_silver` (líneas 163 a 179) y modo de recuperación. Config de las mallas.	Escribir Bronce y Silver base particionados por `LOTE_CARGUE_ID`. El job ya activa la sobrescritura dinámica de particiones, así que solo se reemplaza el lote propio. Cambia la lectura de remediación, Silver y crawlers, así que se planea aparte.
EXT-04	Medio	Si una llave aparece en más de un lote (por ejemplo, porque una corrida se hizo con la exclusión apagada y las llaves se guardaron igual), los lotes de respaldo incluyen los dos.	Silver lee la llave dos veces. En ARL, el `dropDuplicates` sin orden elige una al azar (ARL-02). En otras mallas llega duplicada al SCD (dimensión de cambio lento) (SCD-03).	Simulación del flujo (2.1). Código: llaves guardadas siempre que la segmentación está habilitada (línea 1523); lotes tomados de todas las llaves previas (líneas 696 y 765).	Al armar los lotes de respaldo, detectar llaves en más de un lote y detener con una muestra, o quedarse con el lote más reciente.
EXT-05	Medio	Todo el estado del flujo vive en un solo `parametros_extraccion.json` fijo por malla, que cada job reescribe completo.	Una corrida nueva borra el estado de la anterior, y no queda rastro por ejecución. Agrava EXT-01 y ARL-05.	Código y definición de la Step Function.	A mediano plazo, un manifiesto por ejecución. Mientras tanto, guardar `execution_id` y periodo en cada sección y validarlos al leer.
EXT-06	Medio	El job no configura el logger, así que los mensajes informativos (conteos, lotes y exclusiones) no salen en CloudWatch.	Diagnóstico de incidentes como el de 202606.	Código (línea 23) y QA 12-nov.	Fijar el nivel del logger, igual que SCD-07.
EXT-07	Medio	Cada corrida lee todas las llaves guardadas del segmento y las tablas completas de Bronce y Silver para validar la exclusión y la disponibilidad. Las tablas crecen con cada lote añadido.	El costo y la duración de Glue suben con el tiempo.	Código: líneas 692 a 740 y 106 a 174.	Con EXT-03 (partición por lote), leer solo los lotes necesarios.
EXT-08	Bajo	La consulta a Oracle es `SELECT *` con el `WHERE` armado como texto desde la config.	Trae columnas que nadie usa y cambia si el origen agrega columnas.	Código: líneas 278 a 290.	Listar las columnas en la config.
EXT-09	Bajo	El límite de unidades (1.000.000 de filas por columnas) siempre se supera en ARL y solo genera un aviso.	Ruido en el log, sin control real.	Config de ARL y código, línea 988.	Calibrar por tabla o quitarlo.
EXT-10	Bajo	Queda código sin uso (`obtener_lotes_desde_ruta_llaves`) y las fechas usan la hora local del contenedor.	Mantenimiento.	Código.	Limpiar en la migración al repo modular.

Se conserva: la exclusión con validación de respaldo por llave y lote (falla en lugar de excluir sin respaldo), `combinar_lotes_respaldados`, la reutilización de snapshots compartidos, los checkpoints, la reconciliación final de disponibilidad y las métricas de extracción.

4.3 Estructura del código y mantenibilidad

Medido sobre el repo `data-datalake-niif-rmd-ext-estandar-glue-job` y el Silver de ARL. Aplica a los dos archivos auditados y a los demás jobs del repo.

Job	Líneas	Funciones	Líneas de lógica fuera de funciones	Anidamiento máximo	Función más larga
Extracción	1.893	39	898	8 niveles	`aplicar_exclusion_llaves_segmento`, 159 líneas
SCD (dimensión de cambio lento)	552	7	365	3 niveles	`inicializar_gold_vacio`, 33 líneas
Remediación	681	10	421	4 niveles	43 líneas
Validación técnica	810	14	382	3 niveles	43 líneas
Silver ARL	548	8	279	3 niveles	57 líneas

ID	Severidad	Qué pasa	Por qué importa	Corrección
COD-01	Alto	Los jobs son scripts que corren de arriba abajo: la mayor parte de la lógica está fuera de funciones (898 líneas en la extracción, 365 de 552 en el SCD) y no hay un `main`. El ciclo principal de la extracción llega a 8 niveles de anidamiento.	No se puede probar ni reutilizar una parte sin ejecutar el job completo. Para validar este informe hubo que simular Glue, S3 y Oracle enteros. Cada cambio obliga a leer cientos de líneas.	Separar en tres piezas, como pide el lineamiento de Glue del proyecto: transformaciones puras (por ejemplo, clasificar el SCD y armar los lotes de respaldo), adaptadores de lectura y escritura, y un `main` corto. Empezar por lo que se toca en `scd.py`.
COD-02	Alto	El repo no tiene pruebas. En Sonar, `sonar.coverage.exclusions=src/**.py` saca todo el código de la medición de cobertura, y `tox.ini` tiene `fail_under = 0`.	La cobertura que reporta Sonar no mide nada. Incumple el estándar de código seguro (más de 80 %) y el gobierno DevOps NIIF (70 %).	Convertir los escenarios de la simulación (sección 2.1) en pruebas con `pytest` y Spark local. Quitar la exclusión de cobertura y subir el umbral por etapas.
COD-03	Medio	Hay funciones copiadas entre jobs: `cargar_json_s3` y `guardar_json_s3` están en los 5 jobs, con 2 versiones distintas; `_leer_lotes_disponibles` y `_lotes_requeridos_dataset` están en 3; `normalizar_tabla_relativa` y `parse_s3_uri` tienen versiones distintas.	Un arreglo se tiene que repetir en cada copia, y las copias ya divergen.	Moverlas a un módulo común (`src/comun/`) del repo modular, con sus pruebas.
COD-04	Medio	Hay funciones demasiado largas (`aplicar_exclusion_llaves_segmento` tiene 159 líneas; el lineamiento pide menos de 40) e imports dentro de funciones (6 en la extracción, 3 en remediación y 3 en validación).	Difíciles de leer, de probar y de revisar en un PR.	Dividir por responsabilidad: leer llaves previas, validar respaldo y excluir. Subir los imports al inicio del módulo.
COD-05	Medio	El contrato entre jobs (`parametros_extraccion.json`) no tiene esquema ni versión: cada job lee y escribe claves de texto sueltas. Para saber qué lotes leer, `_lotes_requeridos_dataset` prueba hasta 7 claves distintas.	Renombrar una clave rompe otro job sin aviso. La cadena de alternativas hace difícil saber qué dato se usó (EXT-04 y EXT-05 vienen de aquí).	Definir el contrato en un solo módulo común, con lectura y escritura validadas, y documentarlo.

ID	Severidad	Qué pasa	Por qué importa	Corrección
COD-06	Medio	La configuración tiene valores por defecto silenciosos. Si falta la clave, la extracción asume `permitir_reanudacion = True`, `modo_escritura = overwrite` y `excluir_llaves_segmento = False`, entre otros (líneas 988 a 1025).	Un olvido en la config cambia el comportamiento sin avisar. Por ejemplo, la exclusión se apaga sola (ver EXT-04) o se activa la reanudación (ver EXT-01). El lineamiento del proyecto pide fallar si falta un parámetro.	Exigir las claves que cambian el comportamiento y fallar con un mensaje claro si faltan. Dejar valores por defecto solo para opciones de rendimiento.
COD-07	Bajo	Los errores se lanzan como `ValueError` o `RuntimeError` con texto libre (24 en la extracción). Solo el freno usa un código (`SCD_APAGADO_MASIVO`). Los estados (`SUCCESS` , `FAILED`) se escriben como texto en cada job.	Monitoreo no puede clasificar los fallos. La HU de gobierno de ingestas pide una taxonomía de errores.	Usar códigos de error del catálogo del proyecto, como ya hace el freno, y constantes compartidas para los estados.

4.4 En otros componentes: fuera de la corrección del SCD (dimensión de cambio lento)

ID	Componente y repo	Qué pasa	Impacto	Qué se propone	Quién
ARL-02	Silver ARL: `data-datalake-niif-primas-arl-glue-job` , `silver_primas_arl.py` líneas 416 a 459	Después de unir con personas, el job deduplica por póliza y endoso sin un orden. Las personas jurídicas se extraen con consecutivo, así que una persona puede tener varias filas.	El tercero elegido puede cambiar entre corridas. Como el nombre y los datos del tercero cuentan como cambio, Gold abriría versiones sin un cambio real. Por confirmar con datos.	Elegir el registro con un orden fijo, o deduplicar personas antes del cruce.	Desarrollo
ARL-04	Certificación: config `primasArl.json` y job de certificación	La consulta de detalle agrega una fila de encabezado, así que el job nunca detecta "sin datos". El resumen está vacío en la config.	Puede salir un correo de certificación con totales vacíos, como en el incidente.	Ya está dentro de la tarea de notificaciones (T1).	Desarrollo (T1)
ARL-05	Step Function `niif-<amb>-primas-arl` (IaC)	Un fallo, incluido el del freno, termina sin avisar a nadie. No hay tiempos máximos (los jobs tienen 48 horas). El estado vive en un archivo fijo por malla y no hay protección si coinciden dos ejecuciones, por ejemplo la programación y una manual.	Un fallo pasa desapercibido. Dos corridas a la vez pueden pisarse el estado.	Alarma de fallo, tiempos máximos según la duración real y evitar ejecuciones simultáneas por malla.	DevOps y desarrollo
ARL-06	Validación técnica	Con hallazgos (estado FAILED), el flujo sigue por diseño.	Datos con hallazgos pueden llegar a Gold.	Se mantiene como está. Queda registrado para decisión.	Líder técnico y BA

5. Orden de trabajo

Todo el desarrollo de código se entrega a QA en **6 días hábiles**, del martes 17 al martes 24 de noviembre, con tres líneas en paralelo. El detalle día por día está en la sección 7 del informe de negocio.

Línea	Hallazgos
A, proceso de histórico (`scd.py`)	SCD-01 a SCD-10, COD-01 y COD-04 en `scd.py`
B, extracción y remediación	EXT-01 a EXT-10, COD-06
C, base común, calidad y otros repos	COD-02, COD-03, COD-05, COD-07, COD-01 y COD-04 en extracción, remediación y validación, ARL-02, ARL-04 y el ajuste de configuración de ARL-01 y ARL-03

Los días 5 y 6 son para la prueba completa del flujo (local y DEV) y los paquetes por repo.

Fuera del código: - **DevOps:** versionado del histórico (parte de SCD-06), ARL-05 y el ajuste de crawlers por el guardado por lote (EXT-03). - **BA:** las preguntas de la sección 6, a más tardar el jueves 19, porque desbloquean SCD-02 y EXT-02. - **Decisión del líder técnico y BA:** ARL-06.

Los casos reproducidos en la simulación pasan a ser pruebas unitarias en `tests/` del repo.

6. Preguntas

Estas respuestas desbloquean la corrección del problema raíz (SCD-02, ARL-01, ARL-03) y EXT-02:

1. ¿Cada corrida de Primas ARL debe traer todo hasta el periodo (`<=`) o solo el periodo (`=`)?
2. Al cargar un periodo nuevo, ¿los periodos anteriores deben seguir activos en Gold?
3. ¿Qué define el periodo de un endoso: `PERIODO` o la fecha de proceso (FEC_PROCESO_POINT)?
4. ¿Un endoso ya extraído puede cambiar en Oracle (valor, tercero, sucursal) dentro del mismo periodo? Si puede, una nueva corrida debe traer el valor actual.